

Dokumentacja wstępna

Celem projektu jest implementacja interpretera własnego języka ogólnego przeznaczenia. Język ten będzie uproszczeniem języka C.

1. Założenia:

- Język obsługuje typy liczbowe *int* i *float* oraz pozwala na wykonywanie operacji na liczbach.
- Język obsługuje typ *string* i pozwala na konkatenaację ciągów znaków.
- Język obsługuje typ logiczny *bool*.
- Komentarze rozpoczyna znak #.
- Język pozwala na tworzenie zmiennych o statycznym, słabym typowaniu. Domyślnie zmienne mają stałą wartość po jej przypisaniu.
- Język pozwala na tworzenie własnych funkcji o parametrach przekazywanych przez wartość. Funkcje mogą, ale nie muszą zwracać wartości.
- Możliwe jest rekursywne wywołanie funkcji.
- Nie jest możliwe przeciążanie funkcji.
- Język obsługuje instrukcję pętli *while* oraz instrukcję warunkową *if-else*.
- Możliwe jest wypisanie tekstu na ekranie za pomocą funkcji *print()*.
- Język obsługuje struktury i rekord wariantowy (rekord ten ma budowę podobną do struktury, ale zawiera dodatkowe pole informujące o typie aktualnie przechowywanej wartości).
- Każdy program musi zawierać funkcję *main()*.

2. Przykładowy kod:

- prosta funkcja

```
int fun(int a)
[
    a = a + 1;
    return a;
]

mut int a = 2;
mut int b = fun(a);
print(a);
print(b);

# wyświetli się:
# 2
# 3
```

- funkcja z rekursją

```
int silnia(int n)
[
    if (n == 0)
    [
        return 1;
    ]
    else
    [
        return n * silnia(n-1);
    ]
]
```

- funkcja *main()* z pętlą *while()*

```
int main()
[
    int a = 10;
    mut int b = 1;
    b = silnia(3);
    while (b < a)
```

```

    [
        print(b);
        b = b + 1;
    ]
    return 0;
]

```

- konkatencja stringów

```

string x = "hello";
string y = "world";
string z = x + y;

```

- konwersja typu

```

string s = 12.3;
string t = " meters";
string u = s + t;

```

- struktura

```

struct Item
[
    string name;
    mut float value;
]

```

```

Item my_item("bread", 3.45);
my_item.value = 4.56;

```

- rekord variantowy

```

variant[int, string] a;
a = "hello";

```

```

match a
[
    int i
    [
        print("Wartosc typu int: \n");
    ]
]

```

```

        print(i);
    ]
    string i
    [
        print("Wartosc typu string: \n");
        print(i);
    ]
    default
    [
        print("Wartosc innego typu");
    ]
]

```

3. Obsługa błędów

Każdy błąd skutkuje zatrzymaniem programu.

Przykłady błędów:

- brak średnika

```
bool complete = true
```

Komunikat: "Error in line x, column y: missing a semicolon"

- próba zmiany wartości zmiennej o stałej wartości:

```
int x = 5;
x = 3;
```

Komunikat: "Error in line x, column y: cannot modify a const value"

- próba użycia niezainicjowanej zmiennej

```
int x = y;
```

Komunikat: “Error in line x, column y: uninitialized variable”

- dzielenie przez 0

```
int div(int a, int b)
[
    return a/b;
]
```

```
int c = div(2, 0);
print(c);
```

Komunikat: “Error in line x, column y: cannot divide by 0”

- nieprawidłowy typ

```
int a = "hello";
print(a);
```

Komunikat: “Error in line x, column y: incorrect type”

4. Gramatyka

```
digit_not_zero    =    '1' | '2' | '3' | '4' | '5'
                   |    '6' | '7' | '8' | '9';
```

```
digit             =    '0'
                   |    digit_not_zero;
```

```
letter            =    'a' | 'b' | ... | 'z'
                   |    'A' | 'B' | ... | 'Z';
```

```

symbol      =  '\' | \'.' | \',' | \'/' | \'?'
              |  '<' | '>' | ';' | ':' | '!'
              |  '@' | '#' | '$' | '%' | '^'
              |  '&' | '*' | '(' | ')' | '_'
              |  '-' | '+' | '=';

type         =  'bool'
              |  'int'
              |  'float'
              |  'string';

identifier   =  letter, {letter | digit | '_' };

bool         =  'true'
              |  'false';

int          =  '0'
              |  ['-'], digit_not_zero , {digit};

float        =  int, \'.', {digit};

string       =  '\"', {letter | digit | symbol}, '\"';

literal      =  bool
              |  int
              |  float
              |  string;

argument_list =  expression
              |  argument_list, \', expression;

function_call =  identifier, '(', argument_list, ')';

arithmetic_operation =  expression, '+', expression
                       |  expression, '-', expression
                       |  expression, '*', expression
                       |  expression, '/', expression;

comparison   =  expression, '==', expression
               |  expression, '!=', expression
               |  expression, '>', expression
               |  expression, '<', expression
               |  expression, '>=', expression
               |  expression, '<=', expression;

```

```

logical_operation    =    expression, 'and', expression
                        |
                        expression, 'or', expression
                        |
                        '!', expression;

expression           =    literal
                        |
                        identifier
                        |
                        function_call
                        |
                        arithmetic_operation
                        |
                        logical_operation
                        |
                        comparison;

match_case           =    type, identifier, '[', statement_list,
                        ']'
                        |
                        'default', '[', statement_list, '];

match_statement      =    'match', expression, '[', {match_case},
                        '];

field_access         =    identifier, '.', identifier
                        |
                        field_access, '.', identifier;

struct_creation      =    struct_identifier, identifier, '(',
                        argument_list, ')';

struct_identifier    =    identifier;

struct_field         =    ['mut'], type, identifier
                        |
                        struct_identifier, identifier
                        |
                        variant_declaration;

struct_field_list    =    struct_field
                        |
                        struct_field_list, ',', struct_field;

struct_declaration   =    'struct', struct_identifier, '[',
                        struct_field_list, '];

variant              =    'variant', '[', type, {type}, '];

variant_declaration  =    variant, identifier
                        |
                        'variant', '[', (type | variant,
                        {type | variant}, ']', identifier;

return_statement     =    'return', expression, ';';

condition            =    expression;

```

```

while_statement      =   'while', '(', condition, ')',
                        '[' , statement_list, ']' ;

if_statement         =   'if', '(', condition, ')',
                        '[' , statement_list, ']',
                        ['else', '[' , statement_list, ']' ] ;

assignment           =   identifier, '=', expression, ';'
                        |   field_access, '=', expression, ';' ;

variable_declaration =   ['mut'], type, identifier, ['=',
                        expression], ';' ;

parameter            =   type, identifier ;

parameter_list       =   parameter
                        |   parameter_list, ',', parameter ;

statement            =   variable_declaration
                        |   assignment
                        |   if_statement
                        |   while_statement
                        |   return_statement
                        |   variant_declaration
                        |   struct_declaration
                        |   struct_creation
                        |   match_statement ;

statement_list       =   statement
                        |   statement_list, statement ;

function_declaration =   (type | 'void'), identifier, '(',
                        parameter_list, ')', '[' , statement_list,
                        ']' ;

declaration          =   function_declaration
                        |   variable_declaration
                        |   variant_declaration
                        |   struct_declaration ;

program              =   {declaration} ;

```